

2 A computer program will store data about trees.

The user can enter their requirements for a tree and a suitable tree will be selected.

The program is written using object-oriented programming.

The class `Tree` stores data about the trees.

Tree	
<code>TreeName : STRING</code>	stores the name of the tree
<code>HeightGrowth : INTEGER</code>	stores the number of cm the tree will grow each year
<code>MaxHeight : INTEGER</code>	stores the maximum height in cm that the tree will grow
<code>MaxWidth : INTEGER</code>	stores the maximum width in cm that the tree will grow
<code>Evergreen : STRING</code>	stores whether the tree keeps its leaves as "Yes", or loses its leaves as "No"
<code>Constructor()</code>	initialises <code>TreeName</code> , <code>HeightGrowth</code> , <code>MaxHeight</code> , <code>MaxWidth</code> and <code>Evergreen</code> to its parameter values
<code>GetTreeName()</code>	returns the name of the tree
<code>GetGrowth()</code>	returns the number of cm the tree will grow each year
<code>GetMaxHeight()</code>	returns the maximum height in cm that the tree will grow
<code>GetMaxWidth()</code>	returns the maximum width in cm that the tree will grow
<code>GetEvergreen()</code>	returns whether the tree keeps its leaves or loses its leaves

(a) (i) Write program code to declare the class `Tree` and its constructor.

Do **not** declare the other methods.

Use the appropriate constructor for your programming language.

All attributes must be private.

If you are writing in Python, include attribute declarations using comments.

Save your program as **Question2_J24**.

Copy and paste the program code into part **2(a)(i)** in the evidence document.

[4]

- (ii) The get methods `GetTreeName()`, `GetGrowth()`, `GetMaxHeight()`, `GetMaxWidth()` and `GetEvergreen()` each return the relevant attribute.

Write program code for the get methods.

Save your program.

Copy and paste the program code into part **2(a)(ii)** in the evidence document.

[3]

- (b) The text file `Trees.txt` stores data about 9 trees.

The data in the file is stored in the format:

Tree name,Height growth each year,Maximum height,Maximum width,Evergreen

For example, the first row of data is:

Beech,30,400,200,No

The tree is a Beech. It can grow 30cm each year. It has a maximum height of 400cm. It has a maximum width of 200cm. It is **not** evergreen (it loses its leaves).

The function `ReadData()`:

- creates an array of type `Tree`
- reads the data from the file
- raises an exception if the file is **not** found
- creates a new object of type `Tree` for each tree in the file
- appends each object to the array
- returns the array.

Write program code for `ReadData()`.

Save your program.

Copy and paste the program code into part **2(b)** in the evidence document.

[7]

- (c) The procedure `PrintTrees()` takes a `Tree` object as a parameter and outputs the tree's name, height growth each year, maximum height, maximum width and whether it is evergreen.

The output message changes depending on whether it is evergreen.

If it is evergreen, it is in the format:

`TreeName` has a maximum height `MaxHeight` a maximum width `MaxWidth` and grows `HeightGrowth` cm a year. It does not lose its leaves.

If it is **not** evergreen, it is in the format:

`TreeName` has a maximum height `MaxHeight` a maximum width `MaxWidth` and grows `HeightGrowth` cm a year. It loses its leaves each year.

Write program code for `PrintTrees()`.

Save your program.

Copy and paste the program code into part **2(c)** in the evidence document.

[4]

- (d) The main program calls `ReadData()`, stores the return value and calls `PrintTrees()` with the first object in the returned array.

- (i) Write program code for the main program.

Save your program.

Copy and paste the program code into part **2(d)(i)** in the evidence document.

[2]

- (ii) Test your program.

Take a screenshot of the output(s).

Save your program.

Copy and paste the screenshot into part **2(d)(ii)** in the evidence document.

[1]

- (e) The procedure `ChooseTree()` takes an array of `Tree` objects as a parameter.

The procedure prompts the user to input their requirements for a tree.
The user needs to enter:

- the maximum height the tree can be in cm
- the maximum width the tree can be in cm
- whether they want the tree to be evergreen, or **not** evergreen.

A tree meets the requirements if:

- the tree's maximum height is **not** more than the user's input

and

- the tree's maximum width is **not** more than the user's input

and

- the tree matches their evergreen input.

The procedure creates a new array of all the `Tree` objects that meet all the requirements.

The procedure calls `PrintTrees()` for each `Tree` object that meets all the requirements. If there are no trees that meet all the requirements, a suitable message is output.

- (i) Write program code for `ChooseTree()`.

Save your program.

Copy and paste the program code into part **2(e)(i)** in the evidence document.

[6]

- (ii) The procedure `ChooseTree()` needs amending. After the procedure has output the list of trees that meet all the requirements, the procedure needs to:

- take as input the name of one of the trees that the user would like to buy from those that meet all the requirements
- take as input the height of the tree in cm when it is bought
- calculate and output how many years it will take the tree to grow to its maximum height.

For example, the user inputs the tree, Beech. The tree's height is 40 cm when bought. The tree will take 12 years to reach its maximum height of 400 cm.

Write program code to amend `ChooseTree()`.

Save your program.

Copy and paste the program code into part **2(e)(ii)** in the evidence document.

[2]

(iii) Write program code to amend the main program to call `ChooseTrees()`.

Test your program with the following tree requirements:

- a maximum height of 400 cm
- a maximum width of 200 cm
- a tree that is evergreen (does **not** lose its leaves).

When asked for the tree selection, use the following data:

- first tree name entered is 'Blue Conifer'
- starting height is 100 cm.

Take a screenshot of the outputs.

Save your program.

Copy and paste the screenshot into part **2(e)(iii)** in the evidence document.

[2]